

Basit 2D Fizik Motoru Geliştirme: Çarpışma, Constraint ve Stabilité Analizi

Grafik Programlama Dersi Vize Ödevi

1. AMAÇ

Bu ödev ile öğrencilerin, temel fizik motoru bileşenlerini 2B bir simülasyon ortamında geliştirerek gerçek zamanlı fizik tabanlı animasyon ve etkileşim mantığını kavramaları amaçlanmaktadır. Öğrenciler, parçacık/cisim temsili, yerçekimi etkisi, sayısal integrasyon, çarpışma çözümü ve mesafe kısıtları (constraints) gibi bileşenleri bir araya getirerek çalışır bir mini fizik motoru oluşturacaktır.

Ödev sonunda öğrencilerin yalnızca çalışan bir simülasyon üretmeleri değil; aynı zamanda seçilen integrasyon yönteminin kararlılığını, çarpışma tepkisinin fiziksel tutarlılığını ve constraint çözümünün sistem davranışına etkisini analiz etmeleri beklenmektedir. Böylece bu çalışma, klasik bir programlama ödevi olmanın ötesinde küçük ölçekli bir araştırma/prototip geliştirme deneyimi sunacaktır.

2. GİRİŞ

Fizik motorları, bilgisayar grafiği, oyun programlama, robotik, simülasyon ve dijital ikiz uygulamalarında dinamik nesnelerin davranışını hesaplamak için kullanılan temel yazılım bileşenleridir. Basit bir fizik motorunda bile konum, hız, ivme, kuvvet, çarpışma, enerji ve kısıt denklemleri birlikte ele alınır. Bu nedenle fizik motoru geliştirme, hem matematiksel modelleme hem de yazılım mimarisi açısından öğretici bir çalışma alanıdır.

Bu ödevde öğrencilerden 2B düzlemde çalışan, çoklu cisim desteğine sahip, yerçekimi altında hareket eden ve temel çarpışma/kısıt çözümü içeren bir simülasyon sistemi geliştirmeleri istenmektedir. Proje ayrıca, farklı integrasyon yöntemlerinin karşılaştırılması ve sistem kararlılığının yorumlanması yönüyle mini bir bilimsel rapor niteliği de taşımaktadır.

3. TEORİK ALTYAPI

Bu bölümde, ödev kapsamında geliştirilen fizik motorunun temelini oluşturan matematiksel ve hesaplamalı yöntemler sistematik olarak sunulmaktadır. Öncelikle klasik mekaniğin temel denklemleri ve sayısal integrasyon yöntemleri ele alınacak, ardından çarpışma tespiti ve tepkisi, kısıt tabanlı dinamikler, yapısal bozulma modelleri ve enerji analizi konuları işlenecektir.

3.1. Temel Dinamik Model

Her bir cisim için hareket denklemi Newton'un ikinci yasasına göre tanımlanır: $F = m \cdot a$. Toplam kuvvet altında ivme hesaplanır; ivme yardımıyla hız ve konum güncellenir. Bu ödevde en azından yerçekimi kuvveti sisteme dahil edilmelidir. İstenirse sönümlleme veya kullanıcı tanımlı dış kuvvetler de eklenebilir.

- Yerçekimi vektörü: $g = (0, -9.81)$
- Temel durum değişkenleri: konum x , hız v , ivme a , kütle m , yarıçap r
- Zaman ilerletme sabit zaman adımı ile yapılmalıdır: Δt

Bir fizik motorunun temel görevi, klasik mekaniğin dinamik yasalarını zamana bağlı olarak ileriye taşımaktır. Bir parçacık veya rijit cismin hareketi, ikinci mertebeden bir adi diferansiyel denklem (ODE) sistemi olarak ifade edilir:

$$\frac{d^2 \mathbf{x}}{dt^2} = \frac{\mathbf{F}(\mathbf{x}, \mathbf{v}, t)}{m}$$

Bu sistemin analitik çözümü yalnızca basit kuvvet durumlarında mümkündür. Genel durumda sayısal integrasyon yöntemlerine başvurulur. Sayısal çözümleme aşamasında, bu ikinci mertebeden denklem, iki adet birinci mertebeden ODE sistemine indirgenerek Durum Uzayı (State Space) formunda yazılır:

$$\begin{cases} \frac{d\mathbf{x}}{dt} = \mathbf{v} \\ \frac{d\mathbf{v}}{dt} = \frac{\mathbf{F}(\mathbf{x}, \mathbf{v}, t)}{m} \end{cases}$$

Burada $\mathbf{x}$ konum vektörünü, $\mathbf{v}$ hız vektörünü temsil eder. Bu sistemin sayısal olarak çözülmesi, faz uzayında (phase space) bir yörünge takibi anlamına gelir.

3.2. Sayısal Integrasyon

Fizik motorlarında diferansiyel denklemler kapalı formda çözülmek yerine çoğu zaman sayısal integrasyon ile ayrık zaman adımlarında güncellenir. Sayısal integrasyon, sürekli zamanlı hareket denklemlerini ayrık zaman adımlarına (Δt) dönüştürürken bir **ayrıklaştırma hatası (discretization error)** oluşturur. Seçilen yöntemin karakteristiği; simülasyonun yakınsamasını, sayısal sönümlenmesini ve enerji drift'ini belirler.

Hamiltonyen sistemlerde (toplam enerjinin korunduğu konservatif sistemler), dinamikler bir faz uzayı yörüngesi olarak ele alınır. **Liouville Teoremi** gereği, bu sistemlerin faz uzayı hacmi zamanla korunmalıdır.

$$v_{t+1} = v_t + a_t \cdot \Delta t$$

$$x_{t+1} = x_t + v_t \cdot \Delta t$$

Açık Euler yöntemi, faz uzayı hacmini her adımda genişletir. Bu, sisteme her adımda yapay enerji eklenmesi anlamına gelir ve basit bir harmonik osilatörün bile spiral çizerek sonsuza ıraksamasına (patlama) yol açar. Bunun önüne geçmek için Fizik motorlarında en yaygın kullanılan yöntemlerden biri Semi-Implicit Euler (Symplectic Euler) yöntemidir. Hızın önce güncellenip, konum güncellemesinde **yeni hızın** kullanılması kritik bir fark yaratır:

$$v_{t+1} = v_t + a_t \cdot \Delta t$$

$$x_{t+1} = x_t + v_{t+1} \cdot \Delta t$$

Bu küçük değişiklik, yöntemi birinci mertebeden simplektik yapar. Enerji tam olarak korunmasa bile, bir "shadow Hamiltonian" (gölge Hamiltonyen) üzerinde kalarak enerji hatasının belirli bir bandın dışına çıkmasını engeller.

Velocity Verlet yöntemi, hem konum hem de hızın ikinci mertebe doğrulukla ($O(\Delta t^2)$) güncellenmesini sağlayan gelişmiş bir sayısal integrasyon yaklaşımıdır. Bu yöntem, klasik Euler tabanlı yöntemlere kıyasla daha yüksek doğruluk sunar ve özellikle uzun süreli simülasyonlarda daha kararlı sonuçlar üretir. Fizik motorlarında, özellikle enerji korunumu ve stabilite gerektiren durumlarda tercih edilen önemli bir yöntemdir.

Bu yöntemde konum ve hız güncellemeleri aşağıdaki şekilde gerçekleştirilir:

$$x_{t+1} = x_t + v_t \cdot \Delta t + 0.5 \cdot a_t \cdot \Delta t^2$$

$$v_{t+1} = v_t + 0.5 \cdot (a_t + a_{t+1}) \cdot \Delta t$$

Velocity Verlet yönteminin en önemli özelliklerinden biri, konum güncellemesinden sonra yeni kuvvetlerin hesaplanması ve bu yeni ivmenin hız güncellemesinde kullanılmasıdır. Bu sayede hız hesabı, yalnızca mevcut duruma değil, aynı zamanda bir sonraki zaman adımındaki fiziksel koşullara da bağlı olur. Bu yaklaşım, yöntemin doğruluğunu artırırken sayısal hataların birikmesini de azaltır.

Yöntemin bir diğer önemli özelliği zaman tersinirliğine (time-reversibility) sahip olmasıdır. Bu özellik, simülasyonun zaman içinde ileri ve geri çalıştırıldığında aynı fiziksel yolu takip edebilmesini sağlar. Bu durum, özellikle fiziksel sistemlerin daha gerçekçi modellenmesi açısından önemli bir avantaj sunar. Ayrıca, Velocity Verlet yöntemi enerji korunumu açısından oldukça başarılıdır ve uzun süreli simülasyonlarda toplam enerjide meydana gelen sapmalar (energy drift) diğer yöntemlere göre daha düşüktür.

Velocity Verlet yöntemi, moleküler dinamik simülasyonları, astrofiziksel çok cisimli problemler ve parçacık sistemleri gibi yüksek hassasiyet gerektiren alanlarda yaygın olarak kullanılmaktadır. Bununla birlikte, her zaman adımında yeni kuvvetlerin hesaplanmasını gerektirdiğinden, hesaplama maliyeti Euler yöntemine göre daha yüksektir. Bu nedenle gerçek zamanlı uygulamalarda performans ve doğruluk arasında bir denge kurulması gerekmektedir.

Bu ödev kapsamında Velocity Verlet yöntemi, diğer integrasyon yöntemleri ile karşılaştırmalı olarak kullanılabilir. Öğrenciler, bu yöntemi uygulayarak farklı zaman adımlarında sistem davranışını inceleyebilir, enerji korunumu performansını değerlendirebilir ve sayısal stabilite açısından avantajlarını analiz edebilirler. Bu sayede, fizik motorlarının temelini oluşturan sayısal yöntemler hakkında daha derin bir anlayış geliştirmeleri hedeflenmektedir.

Simülasyon sisteminde yüksek yay sabitleri (k) veya birbirine rijit bağlı kısıtların (rigid constraints) bulunması, sistemin stiff (sert) olarak tanımlanmasına yol açar. Bu durum, hareket denklemlerinin özdeğerleri (eigenvalues) arasında büyük bir ölçek farkı yaratır.

Açık (explicit) integrasyon yöntemlerinde, simülasyonun "patlamaması" için zaman adımı Δt , sistemdeki en yüksek frekanslı salınımı yakalayabilecek kadar küçük olmalıdır. Bu durum Courant–Friedrichs–Lewy (CFL) koşulu ile matematiksel olarak ifade edilir:

$$\Delta t < \frac{2}{\omega_{max}}$$

Burada ω_{max} , sistemin en yüksek doğal frekansıdır. Eğer Δt bu eşiği aşarsa, sayısal hata her adımda katlanarak artar ve sistem fiziksel gerçeklikten tamamen kopar.

Mühendislik Çözümleri ve Trade-off (Denge) Analizi

Stiff sistemlerin çözümünde kullanılan yöntemlerin her biri kendi içinde bir ödünleşim (trade-off) barındırır:

1. **Sub-stepping (Alt Adımlama):** Görsel kare hızı (örneğin 60 FPS) sabit tutulurken, fizik motoru her kare içerisinde n adet küçük adım atar ($\Delta t_{physics} = \Delta t_{frame}/n$).
 - **Avantaj:** Enerji korunumunu bozmadan kararlılığı artırır.
 - **Dezavantaj:** Hesaplama maliyeti n kat artar.
2. **Implicit (Kapalı) Yöntemler:**

Implicit Euler gibi yöntemler, v_{t+1} değerini hesaplamak için o anki kuvvetlerin $t+1$ anındaki durumunu çözer. Bu, genellikle bir doğrusal sistem ($Ax = b$) çözümünü gerektirir.

- **Avantaj:** Koşulsuz kararlılık (unconditionally stable) sunar; çok büyük Δt adımlarına izin verir.
- **Dezavantaj:** Sayısal Sönümleme (Numerical Damping) etkisi yaratır. Sistemdeki enerji, fiziksel olmayan bir şekilde "yutulur" ve hareketler olduğundan daha hantal (viskoz) görünür.

3. XPBD (Extended Position-Based Dynamics):

Pozisyon tabanlı bu yaklaşım, stiff kısıtları bir "uyum" (compliance) parametresi olarak ele alır. Bu sayede, implicit yöntemlerin kararlılığını sunarken, matris tersi alma maliyetinden kaçınır ve zaman adımından bağımsız bir sertlik kontrolü sağlar.

3.3. Çarpışma Tespiti

Çarpışma tespiti, iki aşamalı bir mimari ile hesaplama maliyetini $O(n^2)$ 'den $O(n \log n)$ seviyesine indirir. Çarpışma tespiti, hesaplama maliyetini minimize etmek ve zamansal sürekliliği korumak için çok aşamalı bir boru hattı (pipeline) olarak tasarlanır.

3.3.1. Geniş Aşama (Broad Phase)

Amacı, çarpışması imkansız olan nesne çiftlerini hızla elemektir (culling). Nesneler, karmaşık geometrileri yerine hesaplanması basit Sınırlayıcı Hacimler (Bounding Volumes) ile temsil edilir.

- BVH (Bounding Volume Hierarchy): Nesneler, bir ağaç yapısında kümelenir. Kök düğümden başlayarak hacimlerin örtüşmediği dallar doğrudan budanır.
- Dynamic AABB Tree: Dinamik sahnelerde, nesneler hareket ettikçe ağacın kalitesi düşer. Surface Area Heuristic (SAH) kullanılarak, ağacın toplam yüzey alanını (ve dolayısıyla kesişim olasılığını) minimize edecek şekilde düğümler yeniden dengelenir (rebalancing/refitting).

3.3.2. Dar Aşama (Narrow Phase)

Broad phase'den gelen aday çiftler üzerinde, kesin temas verilerini üretmek için yüksek hassasiyetli geometrik testler uygulanır.

- GJK Algoritması: İki konveks nesnenin çarpışma problemi, Minkowski Farkı ($A \ominus B$) kümesinin orijini (0) içerip içermediği problemine indirgenir. GJK, bu küme içinde orijine en yakın noktayı ararken bir simplex (nokta, çizgi, üçgen veya tetrahedron) oluşturur.
- EPA (Expanding Polytope Algorithm): GJK çarpışma tespit ettiğinde, orijini çevreleyen simplex'i bir "polytope" (çokyüzlü) olarak genişletir. Bu genişleme, Minkowski farkı kümesinin sınırlarına ulaşana kadar sürer ve en kısa penetrasyon vektörünü (normal ve derinlik) belirler.

3.3.3. Sürekli Çarpışma Tespiti (Continuous Collision Detection - CCD)

Ayrık zamanlı simülasyonlarda, bir nesne bir karede duvarın önündeyken diğer karede arkasında olabilir (**tunneling**). CCD, bu durumu engellemek için nesnenin zaman içindeki yörüngesini (swept volume) analiz eder.

- **Time of Impact (TOI):** Çarpışmanın gerçekleştiği $t \in [0, \Delta t]$ anıdır. Bu an, mesafe fonksiyonunun kökü olan $f(t) = 0$ denklemiyle bulunur.
- **Sayısal Çözümler ve Performans:**
 - **Bisection:** Kökün bulunduğu aralığı her adımda yarıya indirir. Yavaştır ancak işaret değişimi olduğu sürece köke ulaşmayı garanti eder.

- **Secant:** Fonksiyonun eğimine bakarak köke tahminler yürütür. Bisection'dan çok daha hızlıdır (1.618 yakınsama hızı) ancak fonksiyonun konveks olmadığı durumlarda kökü ıskalayabilir.
- **Newton-Raphson:** Fonksiyonun türevini ($f'(t)$) kullanarak çok hızlı yaklaşır ancak mesafe fonksiyonunun türevi her zaman analitik olarak mevcut olmayabilir.

Simülasyon en az iki tür çarpışmayı desteklemelidir: cisim-zemin çarpışması ve daire-daire çarpışması.

3.4. Çarpışma Tepkisi (Impulse-Based Response)

Çarpışma tespiti (Narrow Phase) gerçekleştikten sonra, elde edilen normal vektörü ($\mathbf{n}$) ve penetrasyon derinliği kullanılarak nesnelerin hız ve konumları güncellenir. Bu süreçte Newton'un Çarpışma Yasası ve Momentumun Korunumu temel alınır.

Çarpışma anında oluşan ani hız değişimi, impuls (J) kavramı ile açıklanır. İmpuls, momentumdaki değişime eşittir:

$$J = \Delta p = m \Delta v$$

İki cismin çarpışma sonrası normal yöndeki bağıl hızının, çarpışma öncesine oranı Restitüsyon Katsayısı (e) ile belirlenir:

- **$e = 1$:** Tam elastik çarpışma (Kinetik enerji korunur).
- **$e = 0$:** Tam inelastik çarpışma (Cisimler birbirine yapışır).

İmpuls büyüklüğü (j), nesnelerin kütleleri ve bağıl hızları cinsinden şu şekilde formüle edilir:

$$j = \frac{-(1 + e) \cdot (\mathbf{v}_{rel} \cdot \mathbf{n})}{\frac{1}{m_1} + \frac{1}{m_2}}$$

Gerçekçi bir simülasyon için normal yönündeki impulsun yanı sıra, temas yüzeyine teğet (t) yönde de bir sürtünme impulsu hesaplanmalıdır. Genellikle **Coulomb Sürtünme Modeli** kullanılır:

$$f_{tangent} \leq \mu \cdot f_{normal}$$

Burada μ , sürtünme katsayısıdır. Teğetsel impuls, bağıl hızın teğet bileşenini sıfırlamaya çalışırken bu sınır değerini (j_{static} veya $j_{kinetic}$) aşamaz.

Sayısal hatalar ve ayrık zaman adımları nedeniyle cisimler birbirinin içine girebilir (penetrasyon). Sadece hız güncellemesi yapmak bu penetrasyonu çözmez, aksine "sinking" (batma) etkisine yol açar.

- **Doğrudan Pozisyon Düzeltmesi:** Cisimler her karede penetrasyon derinliği kadar zıt yönlerde itilir.
- **Baumgarte Stabilizasyonu:** Penetrasyonu çözmek için sisteme yapay bir geri çağırıcı kuvvet eklenir. Pozisyon hatası (C), bir sonraki adımın hız hedefine dahil edilir:

$$\mathbf{v}_{target} = \mathbf{v}_{new} + \frac{\beta}{\Delta t} C$$

Burada β (genellikle 0.1 - 0.2), hatanın ne kadar hızlı sönümleneceğini belirleyen bias parametresidir.

Zemin gibi sonsuz kütleli ($m_2 = \infty$) kabul edilen nesnelerle etkileşimde, denklem sadeleşir.

$$\mathbf{j} = -(1 + e) \cdot m_1 \cdot (\mathbf{v}_1 \cdot \mathbf{n})$$

Zemin çarpışmasında y koordinatı ve hız bileşeni kontrol edilerek cismin yerin altına girmesi engellenir ve enerji kaybı (e) yansıtılır.

Çarpışma tespitinde geometrik örtüşme kontrolü yapılmalı; tepki aşamasında ise konumsal düzeltme ve restitution tabanlı sekme davranışı uygulanmalıdır. Sadece hızın ters çevrilmesi yeterli kabul edilmeyecektir; penetrasyon düzeltmesi de gereklidir.

- Zemin koşulu: $y - r < 0$ için düzeltme ve sekme
- Daire-daire çarpışması: merkezler arası uzaklık < yarıçapların toplamı
- Çarpışma normali, bağlı hız ve restitution katsayısı kullanılmalıdır

3.5. Enerji ve Stabilité Analizi

Bir fizik simülasyonunun sayısal kararlılığı ve fiziksel gerçekliği, sistemin toplam mekanik enerjisinin zaman içindeki değişimi incelenerek nicelleştirilir. İdeal, dış kuvvetlerin (sürtünme vb.) olmadığı bir sistemde toplam enerji korunmalıdır.

Sayısal integrasyon yöntemleri, her adımda sistemin gerçek faz uzayı yörüngesinden sapmasına neden olur. Bu sapma, toplam enerjide yapay bir artış veya azalış olarak gözlemlenir ve **Enerji Drifti** olarak adlandırılır.

Simülasyonun hassasiyeti **Relatif Enerji Hatası** (E_{error}) ile ölçülür:

$$E_{error} = \left| \frac{E_{total}(t) - E_{initial}}{E_{initial}} \right|$$

Kabul Edilebilir Mühendislik Standartları:

- **Gerçek Zamanlı Uygulamalar (Oyunlar):** $E_{error} < 10^{-2}$ (%1). Görsel süreklilik ön plandadır.
- **Mühendislik ve Akademik Simülasyonlar:** $E_{error} < 10^{-6}$. Uzun vadeli yörünge tahmini ve fiziksel analizler için bu hassasiyet kritiktir.

Modern bir fizik motoru şu bileşenlerden oluşur:

1. **Integrator** (zaman güncelleme)
2. **Broad Phase** (uzamsal veri yapısı)
3. **Narrow Phase** (GJK+EPA)
4. **CCD** (TOI hesaplama - isteğe bağlı)
5. **Collision Response** (impulse)
6. **Constraint Solver** (PGS/XPBD)
7. **Failure Model** (kırılma kontrolü)
8. **Renderer** (görselleştirme)

Tasarımda en kritik parametre **doğruluk-hız dengesi (performance-accuracy trade-off)** 'dir.

Parametre / Yöntem	Gerçek Zamanlı (Oyun/VR)	Yüksek Doğruluk (Mühendislik/Bilim)
--------------------	--------------------------	-------------------------------------

Zaman Adımı (Δt)	1/30–1/60 s (Sabit)	1/300–1/1000 s veya Adaptif
Entegrasyon Yöntemi	Semi-Implicit Euler	Velocity Verlet / RK4
Geniş Aşama (Broad Phase)	AABB + Statik BVH	Dinamik SAH-BVH / Octree
Dar Aşama (Narrow Phase)	İlkel (Primitive) Testler	GJK + EPA (Hassas Çözüm)
Sürekli Çarpışma (CCD)	Kritik Nesneler İçin (Ray-cast)	Her Adımda TOI Analizi
Kısıt Çözücü (Solver)	PGS, 3-5 İterasyon (Hız Odaklı)	XPBD veya PGS, 20+ İterasyon
Enerji Hatası Toleransı	%0.1 - %1.0	< %0.0001 (10–6)

Öğrenciler, simülasyonun yalnızca görsel çıktısını değil, fiziksel davranışını da değerlendirmelidir. Bu amaçla toplam kinetik enerji, potansiyel enerji veya uygun görülen toplam enerji ölçütü zaman boyunca izlenmeli; seçilen integrasyon yönteminin enerji drift üretip üretmediği incelenmelidir.

Temel bileşenler ve beklenen uygulama kapsamı

Bileşen	Temel Denklem / Mantık	Zorunluluk	Beklenen Parametreler	Not
Parçacık/Cisim	x, v, m, r durumu	Zorunlu	Konum, hız, kütle, yarıçap	Sabit ve hareketli cisim ayrımı önerilir
Integrasyon	Euler ve/veya Verlet	En az 2 yöntem	Δt , toplam süre	Karşılaştırmalı analiz yapılmalı
Yerçekimi	$F = m g$	Zorunlu	g değeri	Tüm hareketli cisimlere uygulanmalı
Çarpışma	Örtüşme + restitution	Zorunlu	e katsayısı	Konumsal düzeltme beklenir
Constraint	Mesafe korunumu	Zorunlu	rest length, iterasyon	Violation grafiği istenir
Kırılma/Failure	Eşik aşımı ile kopma	Bonus/İleri	threshold	Mini research yönü için güçlü katkı

4. UYGULAMA GEREKSİNİMLERİ

4.1. Zorunlu Özellikler

- 2B bir simülasyon penceresi veya animasyon çıktısı üretilecek; sistem gerçek zamanlı ya da adım adım çalışabilir.
- Her cisim için en az konum, hız, kütle ve yarıçap bilgisi tutulacaktır.
- Yerçekimi etkisi altında hareket uygulanacaktır.
- En az iki integrasyon yöntemi (Semi-Implicit Euler ve Verlet önerilir) kodlanacak ve karşılaştırılacaktır.
- Zemin çarpışması ve daire-daire çarpışması uygulanacaktır.
- En az bir distance constraint sistemi kurulacaktır.
- Program kullanıcıdan senaryo, integrasyon yöntemi veya temel parametreleri seçebilecek şekilde tasarlanmalıdır.

- Simülasyon boyunca enerji ve/veya constraint violation verileri kaydedilecektir.
- Kod modüler olmalı; en azından Body, Integrator, Collision, Constraint ve Main/Scene benzeri bir ayırım bulunmalıdır.

4.2. İsteğe Bağlı (Bonus) Özellikler

Aşağıdaki her bir özellik en fazla +5 puan olarak değerlendirilebilir; toplam bonus puanı +20 ile sınırlıdır.

- Breakable constraints ve basitleştirilmiş göçme (collapse) senaryosu
- Constraint iterasyon sayısına bağlı hata/stabilite analizi
- Interaktif arayüz (tuş/fare ile parametre değiştirme)
- Gelişmiş görselleştirme: hız vektörleri, collision normal çizgileri, debug görünümü
- Export edilen grafikler veya otomatik rapor üretimi

5. TESLİM EDİLECEKLER

1. Çalıştırılabilir Python kaynak kodu (.py) - açıklayıcı yorum satırları ve okunabilir sınıf/fonksiyon yapısı içermelidir.
2. Rapor (PDF veya Word) - aşağıdaki bölümleri içermelidir: kapak sayfası, amaç, yöntem, kullanılan denklemler, program mimarisi, deney senaryoları, sonuç grafikleri, tartışma ve kaynakça.
3. En az 4 adet ekran görüntüsü veya simülasyon anlık görüntüsü.
4. Kısa demo videosu veya GIF
5. Rapor içinde en az şu grafiklerden ikisi bulunmalıdır: enerji-zaman grafiği, constraint violation-zaman grafiği, aktif constraint sayısı-zaman grafiği, farklı Δt sonuç karşılaştırması.

6. DEĞERLENDİRME KRİTERLERİ

Kriter	Puan
Programın doğru çalışması, veri yapısı ve hata yönetimi	15
Temel fizik modelinin ve yerçekiminin doğru uygulanması	15
En az iki integrasyon yönteminin doğru kodlanması ve karşılaştırılması	15
Çarpışma sistemi: zemin + daire-daire + penetrasyon düzeltmesi	15
Constraint sistemi ve iteratif çözüm	15
Analiz kısmı: enerji, stabilite, Δt veya violation yorumları	15
Raporun içeriği, düzeni, şekiller ve teknik anlatım	10
Toplam	100
Bonus özellikler	+20

Not: Bu ödevin amacı yalnızca çalışan bir animasyon üretmek değil, fiziksel modelleme ile sayısal yöntemlerin birlikte ele alındığı küçük ölçekli bir araştırma/prototip geliştirme deneyimi sunmaktır. Görsellik kadar yöntemsel doğruluk ve teknik raporlama da değerlendirilecektir.

